

DM9051 SPI Fast Ethernet Driver - Performance Tuning Guide

Table of Contents

- [1. Overview](#)
 - [2. Driver Architecture](#)
 - [3. Configuration Structure](#)
 - [4. Performance Tuning Parameters](#)
 - [5. Operating Modes](#)
 - [6. Platform-Specific Configurations](#)
 - [7. Performance Optimization Strategies](#)
-

Overview

The DM9051 driver is a Linux network driver for the Davicom DM9051 SPI Fast Ethernet controller. This guide focuses on configuring and tuning the driver for optimal network performance on different hardware platforms.

Key Performance Features

- **SPI Transfer Modes:** Burst mode vs. Alignment mode
 - **Interrupt vs. Polling:** Hardware interrupt handling vs. software polling
 - **Checksum Offload:** Hardware-accelerated checksumming
 - **TX Continuation Mode:** Optimized transmit packet handling
 - **SKB WB Mode:** Memory boundary for WB alignment
-

Driver Architecture

Configuration Hierarchy

```
driver_config (confdata)
├─ interrupt mode (MODE_POLL, MODE_INTERRUPT, MODE_INTERRUPT_CLKOUT)
├─ mid (mode index: MODE_A, MODE_B, MODE_C)
└─ mod[] array (per-mode configurations)
```

- └─ skb_wb_mode (SKB wb mode)
- └─ tx_mode (TX continuation mode)
- └─ checksumming (checksum offload)
- └─ align (SPI transfer configuration)
 - └─ burst_mode (BURST_MODE_FULL vs BURST_MODE_ALIGN)
 - └─ tx_blk (TX block size for alignment mode)
 - └─ rx_blk (RX block size for alignment mode)

Configuration Structure

Main Configuration: `driver_config`

Located in `dm9051.c`

```
struct driver_config {
    const char *release_version;
    int interrupt;           // Operating mode (poll/interrupt)
    int mid;                 // Mode index selector
    struct mod_config mod[MODE_NUM]; // Per-mode configurations
};
```

Mode Configuration: `mod_config`

```
struct mod_config {
    char *test_info;        // Platform description
    int skb_wb_mode;        // SKB wb mode
    int tx_mode;            // TX continuation mode
    int checksumming;       // Checksum offload enable
    struct align_config align; // SPI transfer alignment
};
```

Alignment Configuration: `align_config`

```
struct align_config {
    int burst_mode;         // BURST_MODE_FULL or BURST_MODE_ALIGN
    size_t tx_blk;          // TX block size (bytes)
    size_t rx_blk;          // RX block size (bytes)
};
```

Engineering Configuration: `eng_config`

Located in `dm9051.h`

```
struct eng_config {
    int force_monitor_rxb;           // Monitor RX byte errors
    int force_monitor_rxc;           // Monitor RX count
    int force_monitor_tx_timeout;    // Monitor TX timeouts
    struct {
        unsigned long delayF[POLL_TABLE_NUM]; // Polling delays (HZ units)
        u16 nTargetMaxNum;                    // Max polling iterations
    } sched;
    u64 tx_timeout_us;                  // TX timeout in microseconds
};
```

Performance Tuning Parameters

1. Interrupt Mode Selection (`interrupt`)

Location: `confdata.interrupt` in `dm9051.c`

Options:

- `MODE_POLL` (0): Software polling mode
 - **Use Case:** Platforms without reliable interrupts
 - **Performance:** Lower CPU efficiency, higher latency
 - **CPU Usage:** Higher (continuous polling)
 - **Recommended For:** Debugging, platforms without interrupt support
- `MODE_INTERRUPT` (1): Hardware interrupt mode (REG39H)
 - **Use Case:** Standard interrupt-driven operation
 - **Performance:** Optimal for most platforms
 - **CPU Usage:** Low (event-driven)
 - **Recommended For:** Production systems with reliable IRQ
- `MODE_INTERRUPT_CLKOUT` (2): Interrupt with clock output
 - **Use Case:** Raspberry Pi 3/5 with specific timing requirements
 - **Performance:** Similar to `MODE_INTERRUPT` with timing adjustments
 - **CPU Usage:** Low
 - **Recommended For:** Raspberry Pi 3/5 platforms

Performance Impact: HIGH - Choosing interrupt mode can reduce CPU usage by 50-90% compared to polling.

Configuration Example:

```
const struct driver_config confdata = {  
    .interrupt = MODE_INTERRUPT, // Use hardware interrupts  
    // ...  
};
```

2. SPI Transfer Mode (burst_mode)

Location: `confdata.mod[mid].align.burst_mode`

Options:

- `BURST_MODE_FULL` (1): Full burst mode
 - **Behavior:** Uses `regmap_noinc_read/write` for entire packets
 - **Performance:** Highest throughput, lowest overhead
 - **Requirements:** SPI controller must support burst transfers
 - **Recommended For:** Modern SPI controllers (e.g., BCM2711, BCM2712)
- `BURST_MODE_ALIGN` (0): Alignment mode with block transfers
 - **Behavior:** Splits transfers into blocks of `tx_blk / rx_blk` bytes
 - **Performance:** Lower throughput, more SPI transactions
 - **Requirements:** Compatible with all SPI controllers
 - **Recommended For:** Older or limited SPI controllers, debugging

Performance Impact: VERY HIGH - Full burst mode can improve throughput by 2-4x.

Configuration Example:

```
{  
    .align = {  
        .burst_mode = BURST_MODE_FULL, // Use full burst mode  
        .tx_blk = 0, // Ignored in burst mode  
        .rx_blk = 0, // Ignored in burst mode  
    }  
}
```

3. Alignment Block Sizes (tx_blk , rx_blk)

Location: `confdata.mod[mid].align.tx_blk` and `rx_blk`

Purpose: Block sizes for alignment mode (only used when `burst_mode = BURST_MODE_ALIGN`)

Recommendations:

- **TX Block Size:** 32-64 bytes
 - Larger blocks = fewer SPI transactions = better performance
 - Must be power of 2 for optimal performance
 - Platform-dependent (test different values)
- **RX Block Size:** 64-128 bytes
 - Typically larger than TX due to receive buffer alignment
 - Larger blocks improve receive throughput
 - Must align with packet boundaries when possible

Performance Impact: MEDIUM-HIGH - Optimal block sizes can improve alignment mode performance by 30-50%.

Configuration Example:

```
{
    .align = {
        .burst_mode = BURST_MODE_ALIGN,
        .tx_blk = 64,    // 64-byte TX blocks
        .rx_blk = 128,   // 128-byte RX blocks
    }
}
```

Tuning Tips:

1. Start with platform default values (e.g., 32/64 for RPi5)
2. Test with increasing block sizes (32, 64, 128, 256)
3. Monitor SPI error rates and packet drops
4. Consider SPI controller FIFO size limitations

4. TX Continuation Mode (tx_mode)

Location: `confdata.mod[mid].tx_mode`

Options:

- `FORCE_TX_CONTI_OFF` (0): Standard TX mode
 - **Behavior:** Waits for TX completion before sending next packet
 - **Performance:** Lower throughput, guaranteed packet ordering
 - **Use Case:** Standard operation, compatibility mode
- `FORCE_TX_CONTI_ON` (1): TX continuation mode
 - **Behavior:** Allows pipelined packet transmission
 - **Performance:** Higher throughput, reduced latency

- **Use Case:** High-throughput applications
- **Note:** Requires proper TX FIFO management

Performance Impact: MEDIUM - Can improve TX throughput by 20-40% in high-load scenarios.

Code Reference: See `dm9051_conti_tx()` vs `dm9051_single_tx()`

Configuration Example:

```
{
    .tx_mode = FORCE_TX_CONTI_ON, // Enable TX continuation
    // ...
}
```

When to Use:

- High TX packet rates (> 1000 pps)
- Low latency requirements
- Sufficient TX FIFO space

When to Avoid:

- Troubleshooting packet transmission issues
- Platforms with limited TX FIFO
- Applications requiring strict packet ordering

5. SKB WB Mode (`skb_wb_mode`)

Location: `confdata.mod[mid].skb_wb_mode`

Options:

- `SKB_WB_OFF` (0): Disable wb alignment
 - **Behavior:** Direct packet buffer usage, no padding
 - **Performance:** Lower overhead, faster processing
- `SKB_WB_ON` (1): Enable wb alignment
 - **Behavior:** Pads odd-length packets to even length
 - **Performance:** Slight overhead for padding

Performance Impact: LOW - Minimal performance impact, but prevents potential errors.

Configuration Example:

```
{
    .skb_wb_mode = SKB_WB_ON, // Enable for ARM platforms
    // ...
}
```

6. Checksum Offload (checksuming)

Location: `confdata.mod[mid].checksuming`

Options:

- `DEFAULT_CHECKSUM_OFF (0)`: Software checksumming
 - **Behavior:** Linux kernel handles checksums in software
 - **Performance:** Higher CPU usage for checksum calculation
 - **Use Case:** Compatibility mode, debugging
- `DEFAULT_CHECKSUM_ON (1)`: Hardware checksum offload
 - **Behavior:** DM9051 hardware calculates TCP/UDP/IP checksums
 - **Performance:** Lower CPU usage, higher throughput
 - **Use Case:** Production systems, high-performance applications

Performance Impact: MEDIUM - Can reduce CPU usage by 10-20% and improve throughput by 15-25%.

Code Reference: See `dm9051_ndo_set_features()` for checksum configuration

Configuration Example:

```
{
    .checksuming = DEFAULT_CHECKSUM_ON, // Enable hardware checksum
    // ...
}
```

Benefits:

- Reduced CPU load for checksum calculation
- Higher packet processing rate
- Lower latency for packet transmission

Note: Requires proper network feature flags configuration (see `dm9051_probe()`).

7. TX Timeout (tx_timeout_us)

Location: `engdata.tx_timeout_us` in `dm9051.h`

Purpose: Maximum time to wait for TX completion before timeout

Default: 2100 microseconds (2.1 ms)

Tuning Guidelines:

- **Lower values** (500-1000 μ s): Faster failure detection, more aggressive timeouts
 - Use when TX should complete quickly
 - May cause false timeouts under high load
- **Higher values** (2000-5000 μ s): More tolerant of temporary delays
 - Use for high-load scenarios or slower SPI buses
 - Reduces false timeout errors

Performance Impact: **LOW** - Affects error recovery time, not normal operation.

Configuration Example (in `dm9051.h`):

```
const struct eng_config engdata = {  
    // ...  
    .tx_timeout_us = 3000, // 3 ms timeout  
};
```

8. Polling Delays (`delayF[]`)

Location: `engdata.sched.delayF[]` in `dm9051.h`

Purpose: Delay values for polling mode (in HZ units, typically milliseconds)

Usage: Only active when `interrupt = MODE_POLL`

Configuration:

```
.sched = {  
    .delayF = {0, 1, 0, 0, 1}, // Delays in HZ units  
    .nTargetMaxNum = POLL_OPERATE_NUM, // Number of iterations  
}
```

Tuning Guidelines:

- **Smaller delays** (0-1 ms): Lower latency, higher CPU usage
 - Use for low-latency requirements
 - May cause excessive CPU usage
- **Larger delays** (1-10 ms): Lower CPU usage, higher latency

- Use to reduce CPU overhead
- May miss packets under high load

Performance Impact: HIGH (for polling mode) - Critical for CPU efficiency in polling mode.

Example Configuration:

```
.sched = {
    .delayF = {0, 1, 2, 1, 1}, // Progressive delays
    .nTargetMaxNum = 5, // Use all 5 delay values
}
```

Operating Modes

Mode Selection (mid)

Location: confdata.mid in dm9051.c

Options:

- MODE_A (0): Raspberry Pi 5 (BCM2712) optimized
- MODE_B (1): Raspberry Pi 4 (BCM2711) optimized
- MODE_C (2): Generic Cortex-A processor

Default Configurations:

MODE_A (RPI5 - BCM2712)

```
{
    .test_info = "Test in rpi5 bcm2712",
    .skb_wb_mode = SKB_WB_ON,
    .tx_mode = FORCE_TX_CONTI_OFF,
    .checksumming = DEFAULT_CHECKSUM_OFF,
    .align = {
        .burst_mode = BURST_MODE_ALIGN,
        .tx_blk = 32,
        .rx_blk = 64
    }
}
```

MODE_B (RPI4 - BCM2711)

```

{
    .test_info = "Test in rpi4 bcm2711",
    .skb_wb_mode = SKB_WB_ON,
    .tx_mode = FORCE_TX_CONTI_OFF,
    .checksumming = DEFAULT_CHECKSUM_OFF,
    .align = {
        .burst_mode = BURST_MODE_FULL, // Full burst mode
        .tx_blk = 0,
        .rx_blk = 0
    }
}

```

MODE_C (Generic Cortex-A)

```

{
    .test_info = "Test in processor Cortex-A",
    .skb_wb_mode = SKB_WB_OFF,
    .tx_mode = FORCE_TX_CONTI_OFF,
    .checksumming = DEFAULT_CHECKSUM_OFF,
    .align = {
        .burst_mode = BURST_MODE_FULL,
        .tx_blk = 0,
        .rx_blk = 0
    }
}

```

Platform-Specific Configurations

Raspberry Pi 5 (BCM2712)

Recommended Settings:

```

.interrupt = MODE_INTERRUPT, // or MODE_INTERRUPT_CLKOUT
.mid = MODE_A,

```

Performance Tuning:

1. Start with MODE_A defaults
2. If SPI errors occur, try `BURST_MODE_ALIGN` with larger block sizes
3. Enable checksum offload for better performance

4. Consider `FORCE_TX_CONTI_ON` for high TX rates

Raspberry Pi 4 (BCM2711)

Recommended Settings:

```
.interrupt = MODE_INTERRUPT,  
.mid = MODE_B,
```

Performance Tuning:

1. MODE_B uses full burst mode (optimal for BCM2711)
2. Enable checksum offload
3. TX continuation mode works well on RPi4
4. Monitor SPI bus utilization

Generic ARM Cortex-A Platforms

Recommended Settings:

```
.interrupt = MODE_INTERRUPT,  
.mid = MODE_C,
```

Performance Tuning:

1. Start with MODE_C defaults
2. Test both burst modes
3. Tune block sizes based on SPI controller capabilities

Performance Optimization Strategies

Strategy 1: Maximum Throughput

Goal: Maximize packets per second and bandwidth

Configuration:

```
const struct driver_config confdata = {  
    .interrupt = MODE_INTERRUPT, // Use interrupts  
    .mid = MODE_B, // Or your platform's optimal mode  
    .mod = {
```

```

// ... MODE_B configuration
{
    .skb_wb_mode = SKB_WB_ON,
    .tx_mode = FORCE_TX_CONTI_ON, // Enable TX continuation
    .checksumming = DEFAULT_CHECKSUM_ON, // Enable checksum offload
    .align = {
        .burst_mode = BURST_MODE_FULL, // Full burst mode
        .tx_blk = 0,
        .rx_blk = 0,
    }
}
};

```

Additional Optimizations:

1. Increase SPI bus speed in device tree
2. Disable debug logging in production
3. Tune kernel network stack buffers
4. Use interrupt mode (never polling)

Strategy 2: Low CPU Usage

Goal: Minimize CPU utilization for network processing

Configuration:

```

{
    .interrupt = MODE_INTERRUPT, // Essential - polling uses more CPU
    .checksumming = DEFAULT_CHECKSUM_ON, // Offload checksum to hardware
    .tx_mode = FORCE_TX_CONTI_OFF, // Simpler mode = less CPU
    .align = {
        .burst_mode = BURST_MODE_FULL, // Fewer SPI transactions
    }
}

```

Additional Optimizations:

1. Increase polling delays if using polling mode
2. Disable unnecessary monitoring/debugging
3. Reduce TX queue size for lower memory usage

Strategy 3: Low Latency

Goal: Minimize packet processing delay

Configuration:

```
{
    .interrupt = MODE_INTERRUPT, // Fastest response time
    .tx_mode = FORCE_TX_CONTI_ON, // Reduce TX delays
    .align = {
        .burst_mode = BURST_MODE_FULL, // Fastest SPI mode
    }
}
```

Additional Optimizations:

1. Use interrupt mode (not polling)
2. Reduce TX timeout for faster error recovery
3. Optimize SPI bus speed
4. Tune kernel interrupt handling

Strategy 4: Compatibility/Debugging

Goal: Maximum compatibility and diagnostic capability

Configuration:

```
{
    .interrupt = MODE_POLL, // Works on all platforms
    .tx_mode = FORCE_TX_CONTI_OFF, // Standard mode
    .checksuming = DEFAULT_CHECKSUM_OFF, // Software checksum
    .align = {
        .burst_mode = BURST_MODE_ALIGN, // Compatible with all SPI
        .tx_blk = 32,
        .rx_blk = 64,
    }
}
```

Additional Settings (in `engdata`):

```
.force_monitor_rxb = FORCE_MONITOR_RXB, // Enable error monitoring
.force_monitor_tx_timeout = FORCE_MONITOR_TX_TIMEOUT,
```

Performance Measurement

Key Metrics to Monitor

- 1. **Throughput:** `ethtool -S eth0 | grep bytes`
- 2. **Packet Rate:** `ethtool -S eth0 | grep packets`
- 3. **Errors:** `ethtool -S eth0 | grep error`
- 4. **CPU Usage:** `top` or `htop` while running network traffic
- 5. **Latency:** `ping` measurements

Benchmarking Commands

```
# Measure TCP throughput
iperf3 -c <server_ip> -t 60

# Measure UDP throughput
iperf3 -c <server_ip> -u -b 100M -t 60

# Monitor statistics
watch -n 1 'ethtool -S eth0 | grep -E "(packets|errors|bytes)"'

# Check driver messages
dmesg | grep -i dm9051 | tail -20
```

Summary of Performance Impact

Parameter	Impact	Recommended Value
<code>interrupt</code>	HIGH	<code>MODE_INTERRUPT</code> (production)
<code>burst_mode</code>	VERY HIGH	<code>BURST_MODE_FULL</code> (if supported)
<code>tx_blk / rx_blk</code>	MEDIUM-HIGH	64/128 (alignment mode)
<code>tx_mode</code>	MEDIUM	<code>FORCE_TX_CONTI_ON</code> (high TX)
<code>checksuming</code>	MEDIUM	<code>DEFAULT_CHECKSUM_ON</code> (production)
<code>skb_wb_mode</code>	LOW	<code>SKB_WB_ON</code> (ARM platforms)
<code>tx_timeout_us</code>	LOW	2100-3000 μ s (default)

Quick Reference

Minimum Changes for Best Performance

1. Enable Interrupt Mode:

```
.interrupt = MODE_INTERRUPT,
```

2. Enable Full Burst Mode:

```
.align.burst_mode = BURST_MODE_FULL,
```

3. Enable Checksum Offload:

```
.checksumming = DEFAULT_CHECKSUM_ON,
```

4. Enable TX Continuation (if high TX rate):

```
.tx_mode = FORCE_TX_CONTI_ON,
```

These four changes typically provide 80-90% of the maximum performance improvement.

References

- Driver source code: `dm9051.c`
- Header definitions: `dm9051.h`
- Register definitions: DM9051 datasheet
- Linux kernel SPI framework documentation
- Linux network driver development guides

Document Version: 1.0

Last Updated: 2025-11-20

Driver Version: r2503_v3.9.1_R2